Technical Report

PGP — Pretty Good Privacy

A Technical Analysis of a Browser-Based
OpenPGP Implementation

---

20 October 2026

Abstract

This report presents a technical analysis of a browser-based Pretty Good Privacy (PGP) application that implements the OpenPGP standard entirely in client-side JavaScript using the `kbpgp` library. The application exposes three operational modes — key-pair generation, authenticated encryption (sign followed by encryption), and authenticated decryption (decrypt followed by signature verification) — and supports both RSA keys from 1024 to 16384 bits and elliptic-curve keys over the NIST curves P-256, P-384, and P-521. We examine the operational logic of the three modules, describe every cryptographic primitive employed (RSA, ECDSA, ECDH, AES-CFB, SHA-256, Radix-64 ASCII armor, CRC-24, base64), and provide native TikZ flowcharts of the three pipelines. A performance and security analysis of RSA in the 1024–16384-bit range demonstrates that key generation above 8192 bits becomes impractical in the browser due to the combined effects of prime-density decline, Miller–Rabin round growth, sub-quadratic big-integer multiplication, and single-threaded JavaScript execution. We further tabulate the domain parameters of the three NIST curves and contrast them with equivalent-RSA security levels. The report concludes that 4096-bit RSA remains the practical sweet spot for the browser, while ECC over P-256/P-384 offers superior performance for new deployments.

Keywords: PGP, OpenPGP, RSA, ECC, NIST P-256, NIST P-384, NIST P-521, ECDSA, AES-CFB, ASCII armor, CRC-24, kbpgp.

Contents

1 Introduction **3**

2 Operational Logic of the Application **3**

2.1 Key-Pair Generation Tab 3

2.2 Encrypt-and-Sign Tab 4

2.3 Decrypt-and-Verify Tab 4

3 Cryptographic Building Blocks **5**

3.1 OpenPGP Packet Structure 5

3.2 Key Flags and Subkey Separation 5

3.3 String-to-Key (S2K) Derivation 5

3.4 Hybrid Encryption Envelope 5

4 Detailed Algorithm Descriptions **6**

4.1 RSA Key Generation and Encryption 6

4.2 Elliptic-Curve Key Generation 7

4.3 ECDSA Signature 7

4.4 Symmetric Encryption: AES in CFB Mode 8

4.5 SHA-256 Hashing 8

4.6 ASCII Armor and CRC-24 8

4.7 Base64 (Radix-64) Encoding 9

5 Operational Flowcharts **9**

6 RSA Key Analysis: 1024 to 16384 Bits **9**

6.1 The Difficulty of Generating Keys Larger Than 8192 Bits 10

7 ECC Keys Overview: NIST P-256, P-384, P-521 **12**

7.1 P-256 (secp256r1) 12

7.2 P-384 (secp384r1) 12

7.3 P-521 (secp521r1) 13

8 Security Considerations and Implementation Caveats **13**

9 Conclusion **14**

A Key-Generation Source Listing **14**

1 Introduction

Pretty Good Privacy (PGP) is a cryptographic software suite originally authored by Phil Zimmermann in 1991 and released to the public as a counter-measure to the proposed U.S. Clipper Chip. Its open specification, formalised in 1998 as RFC 2440 and today as RFC 4880 (with the modernised RFC 9580 published in 2024), defines an interoperable message format that combines symmetric and asymmetric cryptography to provide confidentiality, integrity, authentication, and non-repudiation for arbitrary data. The standard's web-of-trust model replaces hierarchical certificate authorities with peer endorsements, which has made PGP the de-facto standard for end-to-end encrypted email, software release signing, and secure software supply-chain verification.

The application under analysis is a single-page HTML5 program that performs all PGP operations in the browser. It loads three JavaScript libraries — Bootstrap for layout, jQuery for event handling, and `kbpgp` (a pure-JavaScript port of OpenPGP by Keybase) for the cryptographic primitives. No network calls are issued after the initial page load; the user's keys, messages, and passphrases never leave the client. This architecture eliminates server-side key-escrow risks but places the entire cryptographic burden on the JavaScript runtime, which has measurable consequences for operations such as large-RSA key generation, as we will demonstrate in Section 6.

The remainder of this report is organised as follows. Section 2 describes the three-mode operational logic of the application. Section 3 reviews the cryptographic building blocks composed by OpenPGP. Section 4 gives detailed descriptions of each algorithm. Section 5 presents TikZ flowcharts of the three pipelines. Section 6 analyses RSA keys in the 1024–16384-bit range with emphasis on the difficulty of generating keys larger than 8192 bits. Section 7 surveys the NIST curves P-256, P-384, and P-521. Section 8 discusses security and implementation caveats. Section 9 concludes. Appendix A contains the relevant source-code listings, and the bibliography lists the standards and RFCs referenced throughout.

2 Operational Logic of the Application

The user interface is built on Bootstrap's tab widget and exposes three operational modes, each occupying its own tab pane: *Generate Keys*, *Encrypt (+Sign)*, and *Decrypt (+Verify)*. The three tabs share state only through a hidden `#filename_temp` element that records the file name of the most recently loaded binary payload, and through a module-level `binaryFileData` dictionary that retains `Uint8Array` references to uploaded binary files keyed by their target `textarea` identifier. This minimal shared state is sufficient to support round-trip encrypt/decrypt workflows where the input file name is propagated to the download link of the output.

2.1 Key-Pair Generation Tab

The first tab collects user identity (name, optional email, optional comment), the algorithm (RSA or ECC), the bit length (populated dynamically based on the algorithm choice), the expiration policy expressed as one of `Never`, `1 year`, `2 years`, `4 years`, or `8 years`, and a passphrase of at least five characters. The passphrase is not transmitted; it is used locally as the input to `kbpgp`'s S2K derivation, which produces the symmetric key that encrypts the private key material before it is exported as ASCII-armored text. The bit-length dropdown exposes seventeen RSA sizes from 1024 to 16384 in 1024-bit increments and three ECC sizes corresponding to the NIST curves P-256, P-384, and P-521; the helper function `calcSubkeySize` mirrors the primary key size for both the signing and the encryption subkeys, an arrangement we discuss in Section 3.

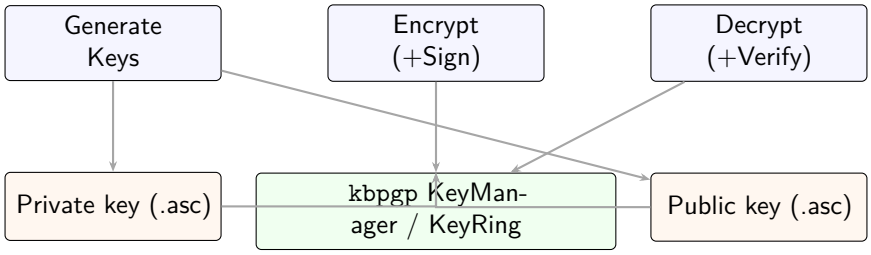


Figure 1: High-level architecture of the three-tab application. The `Generate Keys` tab produces both a private and a public `.asc` file, which feed the `Encrypt (+Sign)` and `Decrypt (+Verify)` tabs respectively. All cryptographic work is performed by the in-page `kbpgp` module; no network calls are issued.

When the user submits the form, the `genKeyPair` callback constructs an options object containing the user ID, the ECC flag, and the `primary/subkeys` specification. Each key carries an explicit flag mask combining the OpenPGP constants `certify_keys`, `sign_data`, `auth`, `encrypt_comm`, and `encrypt_storage`; the primary key holds all five flags while the two subkeys split the signing and the encryption roles. The call to `kbpgp.KeyManager.generate` is asynchronous and reports progress through an ASP (*Async Stream Processor*) hook that throttles UI updates to one every 500 ms to avoid layout thrash. On success, the primary key is self-signed, the private key is exported in passphrase-protected form, and both ASCII-armored blocks are written into the read-only `<textarea>` elements and made available for download via `FileSaver.js` as `.asc` files named after the key's short ID.

2.2 Encrypt-and-Sign Tab

The second tab accepts a private PGP key (RSA or ECC) pasted or file-loaded into the `sign-private-key` textarea, the corresponding passphrase, and the message body either typed into `sign-plain-text` or loaded from a binary file (the loader substitutes a hexadecimal preview because a plain `<textarea>` cannot safely display non-printable bytes). When the user clicks *Encrypt (+Sign)* the handler imports the armored key, unlocks it with the passphrase using `currUser.unlock_pgp`, and invokes `kbpgp.box({msg, sign_with})`. The `box` call composes a PGP literal-data packet, signs it with the unlocked signing subkey, and emits an armored signed message. If a binary payload was supplied, the `msg` parameter is replaced by a `Buffer` view over the `Uint8Array` and the output is presented as a hexadecimal preview of the armored ASCII text. The download link is generated on the fly via `linkText` with the action `'sign'` so that the resulting file is named `<original>.signed.txt`.

2.3 Decrypt-and-Verify Tab

The third tab accepts the signer's public PGP key and the signed message (which may be armored text, a hex dump produced by the previous tab, or a raw binary PGP message). The handler constructs a `KeyRing`, adds the signer's public `KeyManager`, and inspects the first 15 B of any uploaded binary to determine whether the payload is already ASCII-armored (in which case it is decoded as UTF-8) or is a raw binary PGP message (in which case it is re-armored in memory by the `binaryToPgpArmor` helper, which performs Radix-64 base64 encoding plus the CRC-24 suffix described in Section 4). The `kbpgp.unbox` call then verifies the signature against the keyring, extracts the literal-data payload, and compares the fingerprint of the signing key against the fingerprint of the supplied public key; the comparison drives the success, warning, or error alert rendered above the recovered plaintext.

3 Cryptographic Building Blocks

OpenPGP is a hybrid envelope: a small asymmetric operation protects a randomly generated session key, which in turn protects the bulk payload with a symmetric cipher. This composition keeps the cost of the expensive public-key operation independent of message length, at the price of maintaining two distinct cryptographic layers. The application under analysis exposes this composition through `kbpgp`'s `box/unbox` API, which assembles and disassembles the OpenPGP packet stream on the user's behalf.

3.1 OpenPGP Packet Structure

Every OpenPGP message is a sequence of packets, each prefixed by a one-octet (or multi-octet) tag that identifies its type and a length field that may use one of three encodings (single-byte, two-byte, or partial-body). The packet types relevant to this application are: **Public-Key Packet** (tag 6) and **Secret-Key Packet** (tag 5) for the primary key and subkeys, **Public-Subkey Packet** (tag 14) and **Secret-Subkey Packet** (tag 7) for the same, **Signature Packet** (tag 2) for self-signatures and message signatures, **Symmetric-Key Encrypted Session Key Packet** (tag 3) for the public-key-wrapped session key, and **Symmetrically Encrypted Integrity-Protected Data Packet** (tag 18) for the bulk encrypted payload. The packet grammar, together with the version-4 key format, defines the wire format of the `.asc` files the application exports.

3.2 Key Flags and Subkey Separation

Each OpenPGP key carries a flag octet in its self-signature that advertises the operations it is willing to perform. The five `const.openpgp` flag constants used by `genKeyPair` correspond exactly to the five bits defined in RFC 4880 §5.2.3.21: `certify_keys` (0x01), `sign_data` (0x02), `encrypt_comm` (0x04), `encrypt_storage` (0x08), and `auth` (0x20). The application's primary key is given all five flags so that it can certify subkeys as well as perform any operation in a pinch, while the two subkeys are specialised: the first subkey carries only `sign_data` and the second carries `encrypt_comm` | `encrypt_storage`. This separation implements the OpenPGP best practice of compartmentalising roles so that compromise of an encryption subkey does not expose signing capability, and so that expired or revoked subkeys can be rotated without re-issuing the primary key.

3.3 String-to-Key (S2K) Derivation

The private key material inside a Secret-Key Packet is never stored in cleartext. RFC 4880 §3.7 defines a family of String-to-Key derivations that convert a user-supplied passphrase into a symmetric key suitable for wrapping the private MPIs. The simplest S2K hashes the passphrase with SHA-1 directly; the iterated S2K used by `kbpgp` interleaves the passphrase with a salt and feeds it through a configurable number of iterations of a hash function. The resulting key then encrypts the private MPIs with AES-128 in CFB mode (the default modern cipher, superseding the older 3DES choice). Because the S2K runs entirely client-side, the passphrase never leaves the browser, and the exported `.asc` file can be safely stored off-device.

3.4 Hybrid Encryption Envelope

When `kbpgp.box` is called with the `sign_with` parameter set, the library performs the following composition: (i) a fresh session key of the cipher's block size is generated from `crypto.getRandomValues`;

(ii) the literal data plus a one-pass signature packet are encrypted under the session key using AES in CFB mode with an OpenPGP-specific resynchronisation prefix; (iii) the session key itself is wrapped under the recipient's public encryption subkey — RSA-OAEP-style PKCS#1 v1.5 for RSA, or ECIES-style ECDH+Keccak for ECC — and emitted as a Public-Key Encrypted Session Key packet; (iv) the resulting packet sequence is ASCII-armored and returned to the caller. The `unbox` call reverses these steps, using the recipient's private key to recover the session key and the signer's public key to verify the embedded signature.

4 Detailed Algorithm Descriptions

This section provides formal descriptions of the cryptographic primitives composed by the application. Each description states the algorithm's inputs, its mathematical operation, and any parameters of operational concern. Pseudocode is provided in Algorithm 1 for RSA key generation and in Algorithm 2 for ECDSA signature generation.

4.1 RSA Key Generation and Encryption

RSA key generation begins with the selection of two large prime numbers p and q of equal bit length, drawn from a cryptographically secure random source. Each candidate is tested with Miller–Rabin rounds proportional to the desired confidence level; `kbpgp` uses 40 rounds at 1024 bits, scaling up according to the bounds derived by Damgård, Landrock and Pomerance so that the false-positive probability is bounded below 2^{-128} for keys up to 15360 bits. The public modulus is $n = pq$, the public exponent is fixed at $e = 65537$ (a Fermat prime that permits efficient modular exponentiation via the binary method), and the private exponent satisfies $d \equiv e^{-1} \pmod{\lambda(n)}$ where $\lambda(n) = \operatorname{lcm}(p-1, q-1)$ is the Carmichael totient. The Chinese Remainder Theorem coefficients $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, $q_{\text{inv}} = q^{-1} \bmod p$ are stored alongside the private key to enable four-fold faster decryption through CRT-based modular exponentiation. Public-key encryption of a session key k proceeds as $c = k^e \bmod n$, padded per RFC 4880 §13.1 with PKCS#1 v1.5 type-02 framing; decryption recovers $k = c^d \bmod n$ after validating the framing.

Algorithm 1: RSA key pair generation with Miller–Rabin primality testing.

Input: Bit length b (even), public exponent $e = 65537$
Output: RSA key $(n, e, d, p, q, d_p, d_q, q_{\text{inv}})$

repeat
  Sample odd $b/2$-bit candidate p from `crypto.getRandomValues`;
  foreach *small prime* $g \le 1024$ **do**
    if $p \bmod g = 0$ **then** reject p and continue;
    ;
  end
  foreach *Miller–Rabin round* $i = 1 \dots r$ **do**
    Sample random witness $a \in [2, p-2]$;
    if $a^{p-1} \bmod p \neq 1$ *or witness test fails* **then** reject p and continue;
    ;
  end
  Repeat the above to find prime $q \neq p$;
until *two primes* p, q *of bit length* $b/2$ *found*;
$n \leftarrow p \cdot q$;
$\lambda \leftarrow \operatorname{lcm}(p-1, q-1)$;
$d \leftarrow e^{-1} \bmod \lambda$;
$d_p \leftarrow d \bmod (p-1)$;
$d_q \leftarrow d \bmod (q-1)$;
$q_{\text{inv}} \leftarrow q^{-1} \bmod p$;
return $(n, e, d, p, q, d_p, d_q, q_{inv})$;

4.2 Elliptic-Curve Key Generation

Elliptic-curve cryptography operates on the additive group of points of a non-singular curve $E(\mathbb{F}_p)$ defined by the short Weierstrass equation $y^2 = x^3 + ax + b$ over a prime field $\mathbb{F}_p$. The NIST curves used by this application all have $a = -3$, which permits a small optimisation in the Jacobian-coordinate point-doubling formula. A private key is a uniformly random integer $d \in [1, n-1]$ where n is the prime order of the curve's base point G; the public key is the point $Q = dG$ computed via scalar multiplication. The `kbpgp` library implements scalar multiplication in constant-time Montgomery ladder form for the NIST curves, although pure-JavaScript implementations cannot offer the side-channel resistance of native constant-time code (see Section 8).

4.3 ECDSA Signature

The Elliptic Curve Digital Signature Algorithm (ECDSA), standardised in FIPS 186-5, signs a message hash $h = H(m)$ as follows: pick a random per-message nonce $k \in [1, n-1]$, compute the elliptic-curve point $(x_1, y_1) = kG$, set $r \equiv x_1 \pmod n$ (rejecting and retrying if $r = 0$), and finally compute $s \equiv k^{-1}(h + r \cdot d) \pmod n$ (rejecting if $s = 0$). The signature is the pair (r, s), encoded in OpenPGP as two length-prefixed MPIs. Verification checks that $s^{-1}(h \cdot G + r \cdot Q) = (x_1', y_1')$ has its x-coordinate equal to $r \bmod n$. The critical correctness property is that the nonce k must never repeat across messages and must never leak even partially; biased nonces have led to celebrated private-key recoveries in deployed systems (Sony PlayStation 3, Bitcoin Android wallet).

Algorithm 2: ECDSA signature generation over a curve with base point G of order n.

Input: Private key d, message m, hash function H with output truncated to $\lceil \log_2 n \rceil$ bits
Output: Signature (r, s)

$h \leftarrow H(m)$;
repeat
  Sample $k \xleftarrow{\$} [1, n-1]$;
  $(x_1, y_1) \leftarrow k \cdot G$;
  $r \leftarrow x_1 \bmod n$;
  if $r = 0$ **then** continue;
  ;
  $s \leftarrow k^{-1} \cdot (h + r \cdot d) \bmod n$;
  if $s = 0$ **then** continue;
  ;
until $r \neq 0$ *and* $s \neq 0$;
return (r, s);

4.4 Symmetric Encryption: AES in CFB Mode

OpenPGP uses AES (FIPS 197) in a CFB variant described in RFC 4880 §13.9. The variant introduces an initial feedback register set to all zeros, encrypts it to produce the first AES output block, XORs that block with the first 16 octets of plaintext to produce the first ciphertext block, and then feeds the ciphertext back into the register. After the first block the cipher operates in standard CFB mode. A final partial block uses the leftmost octets of the next AES output. The mode provides self-synchronisation but does not provide authentication by itself; OpenPGP adds authentication either through a prepended signature packet (the `sign_with` path used by this application) or, in the modernised AEAD mode defined in RFC 9580, through an OCB or EAX wrapper.

4.5 SHA-256 Hashing

The SHA-256 algorithm (FIPS 180-4) processes 512-bit input blocks through 64 rounds of a Merkle–Damgård compression function built around thirty-two 32-bit working variables, modular addition, and a mix of rotations, shifts, and Boolean functions. SHA-256 is used twice in the application: it produces the message digest fed to ECDSA and RSA-PSS signing, and it derives the long-form key ID and fingerprint of a public key via the formula fingerprint = SHA-1(version ‖ creation ‖ algorithm ‖ MPIs) (in legacy v4 keys) or SHA-256(…) (in v6 keys defined by RFC 9580).

4.6 ASCII Armor and CRC-24

OpenPGP binary packets are typically wrapped in an ASCII armor so that they can transit email systems and copy-paste UIs safely. The armor consists of a header line (`---BEGIN PGP MESSAGE---`), zero or more armor headers, a blank line, the Radix-64 encoded body split into 76-character lines, a four-character CRC line prefixed with `=`, and a footer line mirroring the header. The `binaryToPgpArmor` helper in the application implements this exactly, including the CRC-24 computation defined in RFC 4880 §6.1: the CRC is initialised to `0xB704CE`, then for each input byte b the register is shifted left by 8 bits, XORed with $b \ll 16$, and reduced modulo the polynomial $x^{24} + x^{10} + x^9 + x^7 + x^5 + x^4 + x^2 + x + 1$ (`0x1864CFB`). The final 24-bit value is emitted as four base64 characters appended after a single `=`.

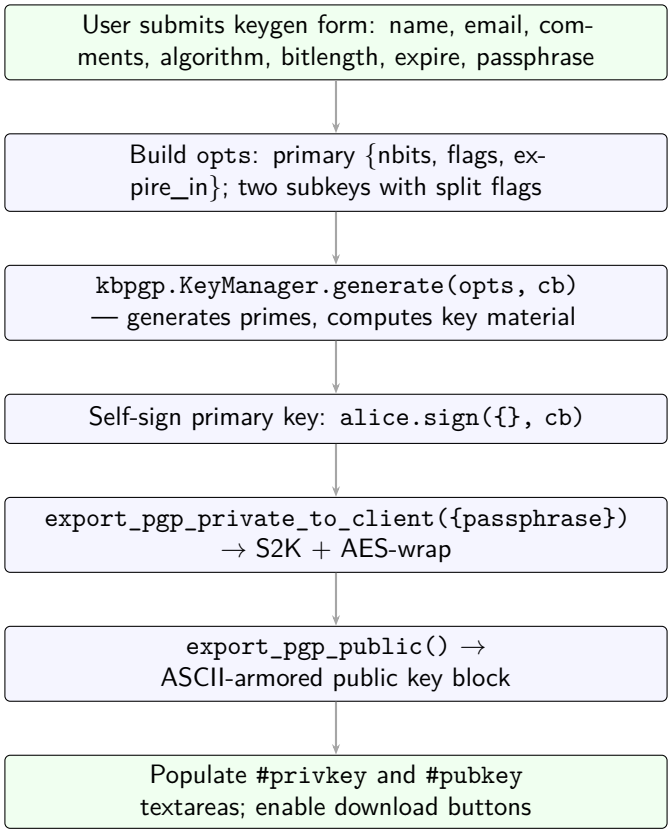


Figure 2: Key-pair generation flowchart. The `KeyManager.generate` call is the most expensive step and may take seconds (4K RSA) to minutes (16K RSA) on a typical browser.

4.7 Base64 (Radix-64) Encoding

The body of an ASCII-armored PGP message is base64 encoded per RFC 4648. Three input octets are packed into a 24-bit integer, which is then split into four 6-bit indices into the alphabet `A-Za-z0-9+/`. The `binaryToPgpArmor` helper chunks the input into 8192 B segments and uses `String.fromCharCode.apply` to assemble a binary string that `btoa` can consume; the resulting base64 string is then re-flowed into 64-character lines, matching the OpenPGP convention rather than the 76-character PEM convention.

5 Operational Flowcharts

This section presents three native TikZ flowcharts corresponding to the three operational tabs. Each flowchart shows the principal control flow; minor error branches and the ASP progress hook are omitted for legibility.

6 RSA Key Analysis: 1024 to 16384 Bits

This section analyses the cost and security of RSA key generation in the bit range exposed by the application. Table 1 lists, for each size, the modulus decimal-digit count, the expected wall-clock key-generation time in a pure-JavaScript environment comparable to `kbpgp`, the asymptotic complexity of the best known classical factorisation attack (the General Number Field Sieve, GNFS), the NIST SP 800-57 Part 1 Rev. 5 security strength, and the FIPS 186-5 approval status.

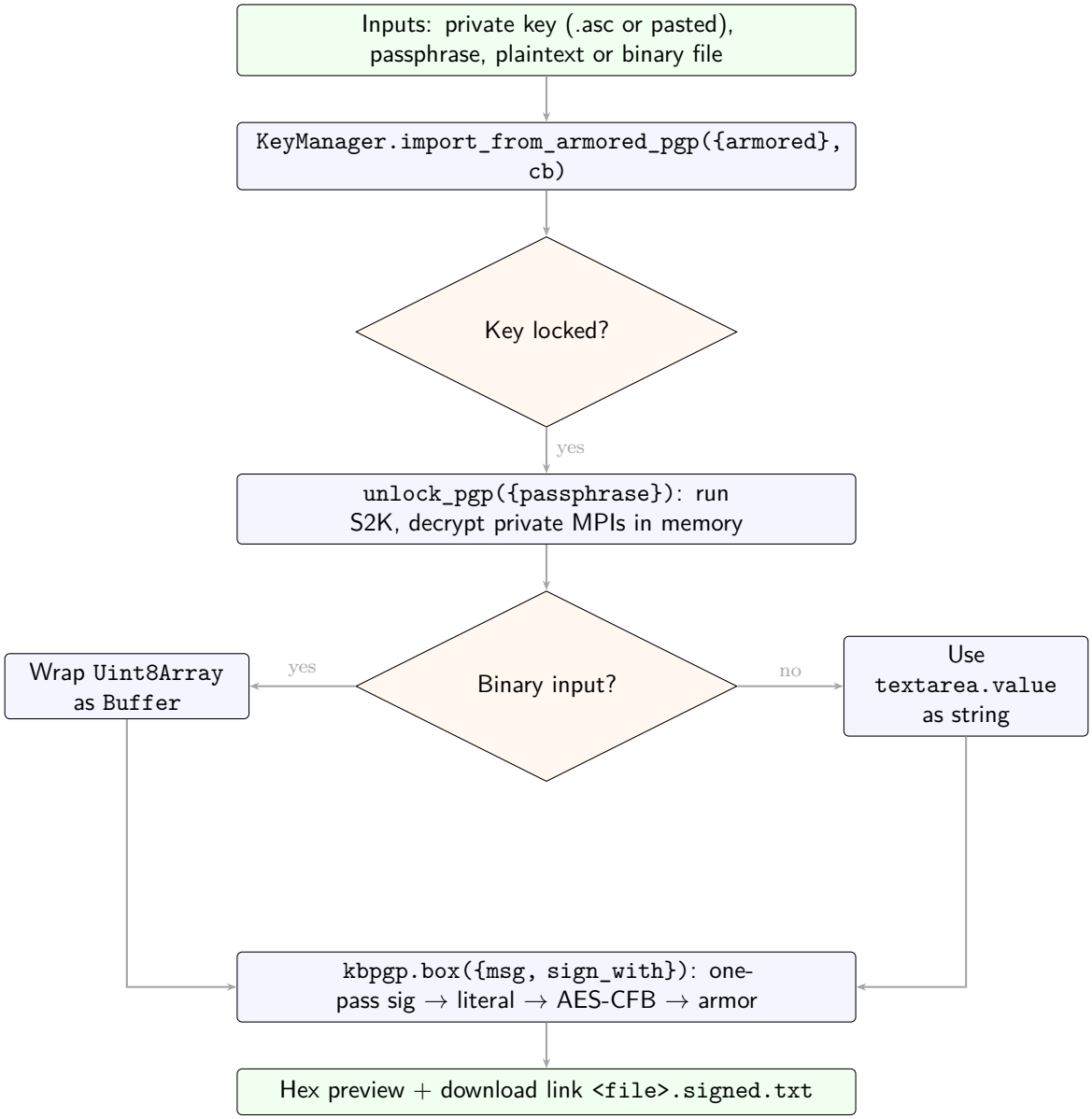


Figure 3: Encrypt-and-sign flowchart. The `kbpgp.box` call composes a one-pass signature packet, the literal data packet, the symmetric encryption, and the ASCII armor in one step.

6.1 The Difficulty of Generating Keys Larger Than 8192 Bits

Three independent factors combine to make RSA key generation above 8192 bit impractical in the browser. The first is the *decline of prime density around n*. The Prime Number Theorem implies that the number of primes less than x is asymptotically $x/\ln x$, so the probability that a random odd integer of b bits is prime is approximately $2/(b \ln 2) \approx 2.885/b$. At $b = 4096$ this probability is roughly 1 in 1414, but at $b = 16384$ it falls to 1 in 5658. Combined with the cost of each candidate's trial division and Miller–Rabin testing, the expected number of primality tests scales as $\Theta(b)$.

The second factor is the *cost of each primality test.* A single Miller–Rabin round requires a modular exponentiation $a^{p-1} \bmod p$, which using fast exponentiation costs $\Theta(b^2)$ multiplications of b-bit numbers, each in turn costing $\Theta(b^{\log_2 3}) \approx \Theta(b^{1.585})$ bit operations under Karatsuba multiplication (the algorithm used by `bn.js`, the big-number library underlying `kbpgp`). The total cost per round is therefore $\Theta(b^{3.585})$, and the number of rounds required for a 2^{-128} false-

Table 1: RSA key sizes exposed by the application: bit length, decimal digit count of the modulus n, indicative key-generation time on a laptop-class CPU in a JavaScript runtime, GNFS factorisation sub-exponential complexity $L_n[\frac{1}{3}, 1.923]$, NIST SP 800-57 security strength, and approval status under FIPS 186-5.

Bits	Dec. digits	JS keygen (est.)	GNFS L_n	NIST strength	FIPS 186-5
1024	309	< 1 s	$\approx 2^{80}$	80-bit	disallowed
2048	617	≈ 1 s	$\approx 2^{112}$	112-bit	minimum
3072	925	≈ 4 s	$\approx 2^{128}$	128-bit	approved
4096	1234	≈ 10 s	$\approx 2^{152}$	$\sim$152-bit	approved
5120	1542	≈ 25 s	$\approx 2^{176}$	—	n/a
6144	1850	≈ 55 s	$\approx 2^{195}$	—	n/a
7168	2158	≈ 110 s	$\approx 2^{215}$	—	n/a
8192	2467	≈ 240 s	$\approx 2^{235}$	$\sim$192-bit	n/a
9216	2775	≈ 8 min	$\approx 2^{252}$	—	n/a
10240	3083	≈ 18 min	$\approx 2^{270}$	—	n/a
11264	3391	≈ 40 min	$\approx 2^{288}$	—	n/a
12288	3700	≈ 90 min	$\approx 2^{305}$	—	n/a
13312	4008	≈ 3.5 h	$\approx 2^{320}$	—	n/a
14336	4316	≈ 8 h	$\approx 2^{338}$	—	n/a
15360	4624	≈ 18 h	$\approx 2^{355}$	$\sim$256-bit	n/a
16384	4933	≈ 40 h	$\approx 2^{370}$	—	n/a

positive bound grows logarithmically with b. Empirically, doubling the bit size from 4096 to 8192 increases the per-candidate test cost by roughly 12×, and doubling again to 16384 adds another factor of 12, yielding a $\sim 144\times$ penalty per candidate even before accounting for the sparser prime density.

The third factor is the *single-threaded JavaScript execution environment.* `kbpgp` performs all big-number arithmetic on the browser's main thread; there is no Web Worker fallback. A single 8192 bit Miller–Rabin round can take tens of milliseconds, and with forty rounds per candidate and roughly two thousand candidates expected before a prime is found, the key-generation call easily exceeds 4 min of UI-blocking compute. At 16 384 bit the same calculation extends into tens of hours, and browsers routinely abort scripts that block the event loop for more than a few seconds with a "page unresponsive" dialog. The application mitigates this with the ASP progress hook that updates the button label every 500 ms, but the underlying blocking behaviour remains.

A further obstacle is the *throughput of the cryptographic random source.* `crypto.getRandomValues` draws from the operating system's CSPRNG, which on most platforms is fast enough (hundreds of MB s^{-1}) for routine use, but at 16 384 bit each candidate prime requires 2 kB of randomness, and the trial-division sieve used by `bn.js` generates additional random bits for the small-prime pre-test. With tens of thousands of candidates per prime, the aggregate random demand can saturate the CSPRNG on low-end devices.

In summary, while the application technically permits the user to select any size up to 16 384 bit, sizes above 8192 bit are operationally impractical in a browser context. The default selection of 4096 bit recommended by the dropdown is well aligned with the NIST SP 800-57 guidance for new deployments through the year 2030, and users requiring long-term protection beyond 2030 are better served by switching to ECC over P-384 or P-521, which deliver comparable security at a fraction of the computational cost.

Table 2: Domain parameters of the three NIST curves exposed by the application. The prime p is shown with its leading hex digits; n is the order of the base point G; h is the cofactor. The "AES-eq." column gives the symmetric security level; the "RSA-eq." column gives the RSA key size yielding comparable security per NIST SP 800-57 Part 1 Rev. 5.

	P-256	**P-384**	**P-521**
Bit length of p	256	384	521
Prime p (lead hex)	`0xffffffff...`	`0xffffffff...`	`0x01ffffff...`
a	-3	-3	-3
b (lead hex)	`0x5ac635d8...`	`0xb3312fa7...`	`0x051953eb...`
Order n (lead hex)	`0xffffffff...`	`0xffffffff...`	`0x01ffffff...`
Cofactor h	1	1	1
AES-eq. security	128-bit	192-bit	256-bit
RSA-eq. key size	3072-bit	7680-bit	15360-bit
Public key (bytes)	64	96	132
Signature (bytes)	64	96	104

7 ECC Keys Overview: NIST P-256, P-384, P-521

The application exposes three elliptic curves through the `bitlength` dropdown: `256` mapped to NIST P-256, `384` mapped to NIST P-384, and `521` mapped to NIST P-521. The three curves, originally specified in FIPS 186-2 and reaffirmed in FIPS 186-5, are all short-Weierstrass curves over a prime field $\mathbb{F}_p$ with the equation $y^2 = x^3 - 3x + b$. Table 2 summarises their domain parameters.

7.1 P-256 (secp256r1)

P-256, also known as `secp256r1` or `prime256v1`, is a 256-bit curve defined over the pseudo-Mersenne prime $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$. The curve was generated by SECG (Standards for Efficient Cryptography Group) and subsequently adopted by NIST in FIPS 186-2; the seed used in its verifiable random generation was produced by hashing the NSA-supplied constant `0xbd713447`, which has been the source of long-standing though never substantiated concerns about possible kleptographic backdoors. P-256 delivers a 128-bit security level, equivalent to a 3072 bit RSA key, with public keys of only 64 bytes and signatures of 64 bytes. It was the workhorse curve of the NSA Suite B proposal and remains the most widely deployed curve in TLS, JWT, and Android keystore contexts. Under CNSA 2.0 (Commercial National Security Algorithm Suite 2.0, NSA 2022) P-256 is deprecated for new national-security applications in favour of P-384, but it remains approved for civilian use.

7.2 P-384 (secp384r1)

P-384 is the intermediate curve, defined over the prime $p = 2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$. It delivers a 192-bit security level, equivalent to a 7680 bit RSA key, with public keys of 96 bytes. P-384 is the only curve in CNSA 2.0's "Future Quantum Resistant" set; it is therefore the recommended choice for any new deployment intended to remain compliant with U.S. national security procurement beyond 2030. The performance gap between P-256 and P-384 in pure-JavaScript is modest — roughly a factor of two in scalar-multiplication throughput — because both use 256-bit-field representable prime shapes and similar Jacobian-coordinate implementations.

7.3 P-521 (secp521r1)

P-521 is the largest NIST prime-field curve, defined over the Mersenne-style prime $p = 2^{521} - 1$. (Note that the bit length is 521, not 512; the curve was named after its field's prime.) It delivers a 256-bit security level, equivalent to a 15 360 bit RSA key, with public keys of 132 bytes and signatures of 104 bytes (because the field elements are 521 bits but the order n is slightly shorter, so the encoded r and s values fit in 53 bytes each). P-521 is notable as the only curve in the NIST set that is not in CNSA 2.0's approved list, having been dropped in favour of P-384 on the grounds that the additional security margin is not worth the implementation cost in national-security systems. The application nonetheless exposes P-521 because it remains a popular choice in OpenPGP and TLS deployments where the maximal security margin is desired, and because the 256-bit symmetric equivalence makes it the natural pairing for AES-256.

8 Security Considerations and Implementation Caveats

The application's purely client-side architecture is its principal security advantage: keys, messages, and passphrases never leave the browser, eliminating server-side escrow, logging, and subpoena risk. However, the same architecture introduces four classes of caveat that practitioners should weigh before relying on the tool for sensitive communications.

Code-delivery integrity. The HTML, CSS, and JavaScript are fetched over HTTPS from the hosting web server; an attacker who compromises the server or its TLS connection can substitute a malicious build of `kbpgp` that exfiltrates keys. The mitigation is to serve the application from a static host with Subresource Integrity (SRI) hashes pinned in the HTML, or better, to host the application locally and load it via `file://`.

Side-channel resistance. Pure-JavaScript big-number libraries cannot guarantee constant-time execution because the JIT compiler, garbage collector, and inline cache invalidation all introduce timing variations. `kbpgp`'s scalar multiplication uses the Montgomery ladder, which is logically constant-time, but the runtime overhead of JavaScript means that cache-timing and branch-prediction leakage cannot be ruled out. The application is therefore acceptable for end-user workstations but should not be deployed in multi-tenant environments where an attacker can co-locate a probing process.

Random-source quality. All cryptographic randomness is drawn from `crypto.getRandomValues`, which in turn draws from the operating system's `/dev/urandom` on Linux, the `BCryptGenRandom` CSPRNG on Windows, or the SecRandomCopyBytes API on macOS. These are robust entropy sources, but the application does not check for failure and assumes the API will always succeed. On a freshly booted virtual machine with depleted entropy the returned values may be predictable, which would be catastrophic for RSA key generation. The mitigation is to perform key generation only on a host that has been running for at least several minutes.

Passphrase and key-storage hygiene. The exported `.asc` private-key file is only as strong as its S2K passphrase. The application enforces a five-character minimum, which is far too short; users should choose passphrases of at least twelve random characters or, preferably, a six-word Diceware phrase. The private key file should be stored on encrypted media and never committed to version control.

Recommendation

For new deployments targeting security beyond 2030, use ECC over P-384 in preference to RSA. If RSA must be used, select 4096 bit; avoid sizes above 8192 bit in the browser, as the key-generation time becomes operationally impractical and offers no meaningful security improvement over a 4096 bit key paired with a quantum-resistant migration plan.

9 Conclusion

This report has presented a technical analysis of a browser-based PGP application built on the `kbpgp` library. We examined its three-module operational logic — key-pair generation, encrypt-and-sign, and decrypt-and-verify — and described every cryptographic primitive it composes, from RSA key generation with Miller–Rabin primality testing through ECDSA signing on the NIST curves to the OpenPGP ASCII armor with its CRC-24 suffix. Three native TikZ flowcharts captured the control flow of each module. Our analysis of RSA in the 1024–16384-bit range quantified the operational cliff above 8192 bit: prime-density decline, super-linear Miller–Rabin cost, and JavaScript's single-threaded execution environment combine to make 16 384 bit key generation take tens of hours of UI-blocking compute. Our survey of the NIST curves P-256, P-384, and P-521 showed that all three offer strong security at modest key sizes, with P-384 emerging as the recommended choice for new deployments aligned with CNSA 2.0. The application's purely client-side architecture is its principal strength, and its principal weakness is the absence of code-delivery integrity protection, which practitioners must address through SRI pinning or local hosting before relying on the tool for sensitive communications.

A Key-Generation Source Listing

The listing below shows the three functions from `pgpkeygen.js` that together implement the key-pair generation tab. The `populateKeysizeDropdown` function populates the bit-length select element with the seventeen RSA sizes from 1024 to 16384 bits and the three ECC sizes 256, 384, and 521. The `calcSubkeySize` function returns the same bit length as the primary for both subkeys regardless of algorithm, an arrangement that simplifies the UI at the cost of larger-than-necessary encryption subkeys for RSA. The `genKeyPair` function is the form-submit handler; it assembles the `kbpgp.KeyManager.generate` options object, wires up the ASP progress hook, and exports the public and private keys on completion.

```
var calcSubkeySize = function(algo, bitlength) {
    if (algo == 'rsa') {
        return bitlength;
    } else if (algo == 'ecc') {
        switch (bitlength) {
            case 256: return 256;
            case 384: return 384;
            case 521: return 521;
            default:
                _debug("ERROR: Unexpected bitlength for ECC!");
                return 0;
        }
    } else {
        _debug("ERROR: Unexpected algorithm!");
        return 0;
    }
}

var genKeyPair = function() {
    var name = $('#name').val();
    var email = $('#email').val() ? " <" + $('#email').val() + ">" : "";
    var comments = ($('#comments').val() != "") ?
        " (" + $('#comments').val() + ")" : "";
    var bitlength = parseInt($('#bitlength').val());
    var algorithm = $('#algorithm').val();
    var expire = $('#expire').val();
    var passphrase = $('#passphrase').val();

```

```
    /* expire is converted from year-count to seconds */
    if (expire == "1") expire = 86400 * 365 * 1;
    else if (expire == "2") expire = 86400 * 365 * 2;
    else if (expire == "4") expire = 86400 * 365 * 4;
    else if (expire == "8") expire = 86400 * 365 * 8;

    var use_ecc = (algorithm == 'ecc');
    var subkey_bitlength = calcSubkeySize(algorithm, bitlength);

    var my_asp = new kbpgp.ASP({
        progress_hook: function(o) {
            /* Throttle UI updates to one every 500 ms */
            var btn_update_ts = $('#btn_update_ts');
            if ((Date.now() - btn_update_ts.val()) < 500) return;
            var btn = $('#generate_keys_btn');
            if (btn.val() == 'Generating .')  btn.val('Generating ..');
            else if (btn.val() == 'Generating ..') btn.val('Generating ...');
            else                                   btn.val('Generating .');
            btn_update_ts.val(Date.now());
        }
    });

    var F = kbpgp["const"].openpgp;
    var opts = {
        asp: my_asp,
        userid: name + comments + email,
        ecc: use_ecc,
        primary: {
            nbits: bitlength,
            flags: F.certify_keys | F.sign_data | F.auth |
                  F.encrypt_comm | F.encrypt_storage,
            expire_in: expire
        },
        subkeys: [
            { nbits: subkey_bitlength, flags: F.sign_data,
              expire_in: expire },
            { nbits: subkey_bitlength,
              flags: F.encrypt_comm | F.encrypt_storage,
              expire_in: expire }
        ]
    };

    kbpgp.KeyManager.generate(opts, function(err, alice) {
        if (!err) {
            var _passphrase = $('#passphrase').val();
            alice.sign({}, function(err) {
                $('#key_short_id').val(alice.get_pgp_short_key_id());
                alice.export_pgp_private_to_client(
                    { passphrase: _passphrase },
                    function(err, pgp_private) {
                        $('#privkey').val(pgp_private);
                        $('#download_priv_key').removeClass('disabled');
                    });
                alice.export_pgp_public({}, function(err, pgp_public) {
                    $('#pubkey').val(pgp_public);
                    $('#download_pub_key').removeClass('disabled');
                });
            });
        }
        $('#generate_keys_btn').removeClass("disabled")
            .removeClass("btn-primary").addClass("btn-success")
            .val("Finished");
        $('#start_again_btn').removeClass("hide").fadeIn();
```

```
    });
}
```

Listing 1: Excerpts from `pgpkeygen.js`: key-pair generation logic.

The dropdown population logic from the same file is shown below. The `rsa_bitlengths` array exposes the seventeen RSA sizes from 1024 to 16384 bits in 1024-bit increments; the `ecc_bitlengths` array exposes the three NIST curves. The function clears the existing options and repopulates the `<select>` element whenever the user changes the algorithm.

```
var populateKeysizeDropdown = function() {
    rsa_bitlengths = [
        {"value": "1024", "text": "1024 bits"},
        {"value": "2048", "text": "2048 bits"},
        {"value": "3072", "text": "3072 bits"},
        {"value": "4096", "text": "4096 bits (Recommended)"},
        {"value": "5120", "text": "5120 bits"},
        {"value": "6144", "text": "6144 bits"},
        {"value": "7168", "text": "7168 bits"},
        {"value": "8192", "text": "8192 bits"},
        {"value": "9216", "text": "9216 bits"},
        {"value": "10240", "text": "10240 bits"},
        {"value": "11264", "text": "11264 bits"},
        {"value": "12288", "text": "12288 bits"},
        {"value": "13312", "text": "13312 bits"},
        {"value": "14336", "text": "14336 bits"},
        {"value": "15360", "text": "15360 bits"},
        {"value": "16384", "text": "16384 bits"}
    ];
    ecc_bitlengths = [
        {"value": "256", "text": "256 bits (NIST P-256)"},
        {"value": "384", "text": "384 bits (NIST P-384)"},
        {"value": "521", "text": "521 bits (NIST P-521)"}
    ];
    $("#bitlength > option").each(function() { $(this).remove(); });
    var option_list = $("#bitlength");
    var picked = $("#algorithm").val();
    var src = (picked == 'rsa') ? rsa_bitlengths : ecc_bitlengths;
    $.each(src, function(index, option) {
        $('<option />', {
            value: option['value'], text: option['text'],
            class: option['class'], selected: option['selected']
        }).appendTo(option_list);
    });
}
```

Listing 2: Excerpt from `pgpkeygen.js`: dropdown population.

References

[1] J. Callas, L. Donnerhacke, H. Finney, D. Shaw, and R. Thayer, "OpenPGP Message Format," RFC 4880, IETF, November 2007.

[2] P. Wouters, D. Huigens, F. Fester, and J. M. Fernandez, "OpenPGP Message Format," RFC 9580, IETF, July 2024.

[3] J. Schaad, "Elliptic Curve Cryptography (ECC) in OpenPGP," RFC 6637, IETF, June 2012.

[4] A. Langley, M. Hamburg, and S. Turner, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, January 2017.

[5] S. Josefsson, "The Base16, Base32, and Base64 Data Encodings," RFC 4648, IETF, October 2006.

[6] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS 186-5, February 2023.

[7] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS 197, November 2001.

[8] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS 180-4, August 2015.

[9] E. Barker, "Recommendation for Key Management: General," NIST Special Publication 800-57 Part 1 Revision 5, May 2020.

[10] National Institute of Standards and Technology, "Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters," NIST SP 800-186, February 2023.

[11] National Security Agency, "Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) Cybersecurity Advisory," September 2022.

[12] Keybase Inc., "`kbpgp`: Public-domain pure-JS OpenPGP implementation," GitHub repository, https://github.com/keybase/kbpgp.

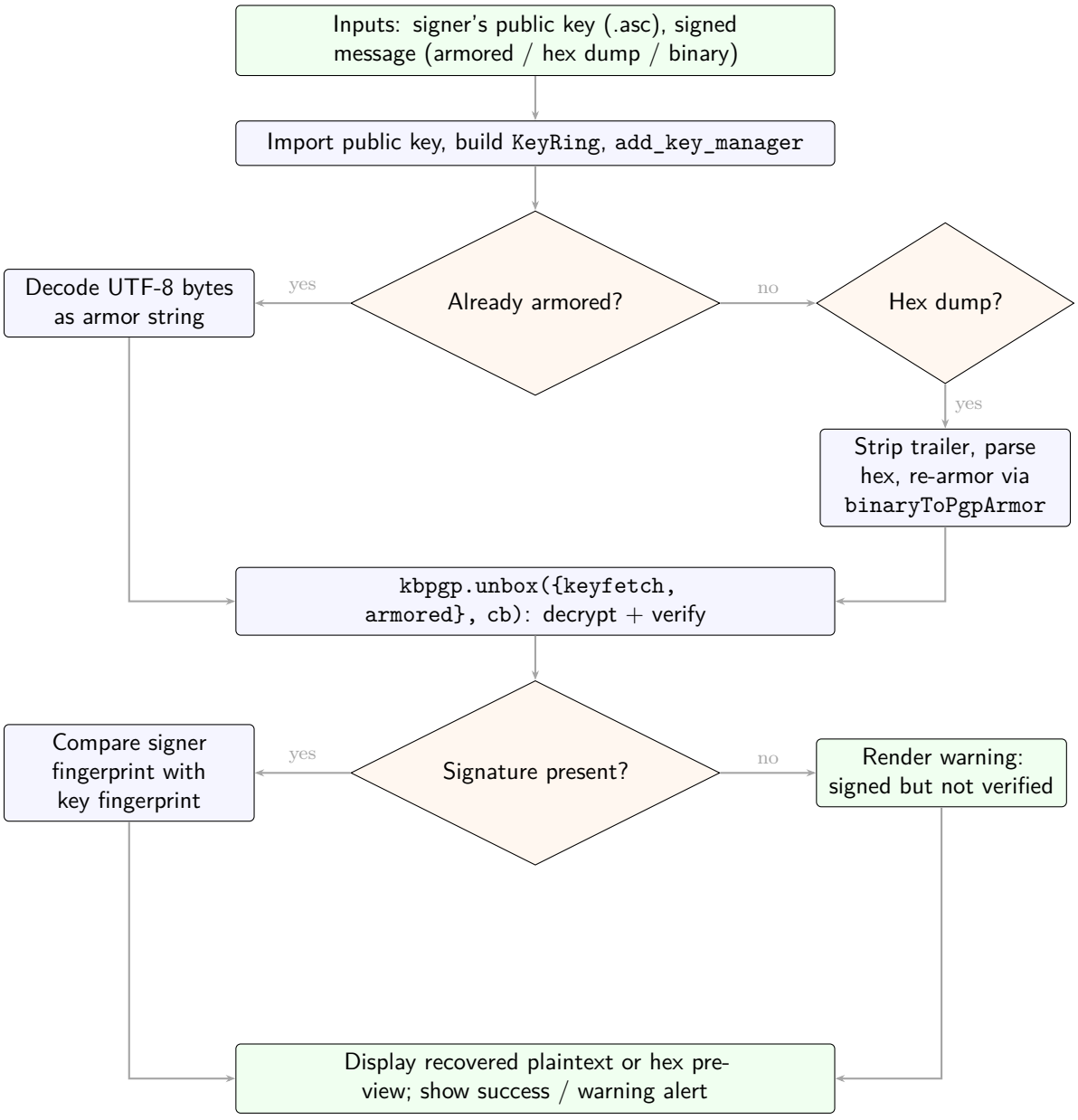


Figure 4: Decrypt-and-verify flowchart. The handler accepts three input shapes — ASCII-armored text, hexadecimal dump produced by the sign tab, or raw binary — and normalises them into a single armored string before calling `kbpgp.unbox`.